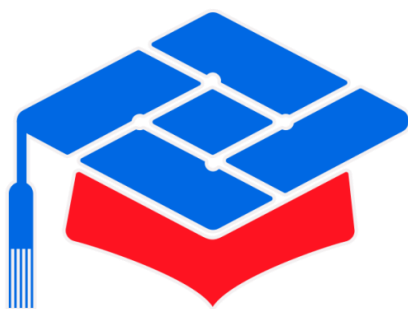




第九届

全国大学生集成电路创新创业大赛

技术报告

报告类型：	技术报告
参赛杯赛：	竞业达杯
作品名称：	全自研 RV32IM 乱序双发射 SoC
队伍编号：	CICC1004534

目录

摘要	1
1 工程内容、功能范围与主要成果	1
1.1 工程目录与生成文件	1
1.2 功能实现范围	1
2 SoC 总体结构与两路时钟	2
2.1 时钟与复位	2
2.2 150 MHz 系统与 50 MHz 测温显示电路的隔离	3
3 乱序双发射处理器结构	3
3.1 结构参数	3
3.2 乱序执行和按顺序收尾	4
3.3 退休结果输出	4
4 取指、分支预测、寄存器分配与恢复	5
4.1 取指和旧响应丢弃	5
4.2 分支预测	5
4.3 寄存器分配和分支恢复记录	5
5 执行单元、读写内存与缓存	6
5.1 EX0/EX1 分工	6
5.2 读写内存接口与请求队列	6
5.3 指令缓存、数据缓存与主存	7
6 指令、系统寄存器、异常与硬件加速	7
6.1 已实现的指令	7
6.2 系统寄存器与中断	7
6.3 CoreMark 自定义加速	8
7 RT-Thread Nano 3.1.5 移植实现	8
7.1 移植范围与源码组成	8
7.2 复位入口与 64 KiB 内存布局	9
7.3 内核启动顺序与线程建立	10
7.4 线程栈与上下文切换	11
7.5 定时中断与抢占调度	11
7.6 FinSH/MSH 注册、输入与退格编辑	13
7.7 MSH 命令与 CoreMark 工作线程	14
7.8 固件编译与 BRAM 初始化	15
8 外设功能设计与传感器显示	16
8.1 地址解码与总线响应	16
8.2 UART 115200 8N1	16
8.3 周期定时器、外部中断与 CoreMark LED	17
8.4 DS18B20 电气连接与管脚约束	18
8.5 1-Wire 状态机与微秒时序	19
8.6 Scratchpad 校验与温度发布	19
8.7 温度换算与 8 位 SEG 扫描	20
8.8 同一 bit 文件内的功能整合	21
9 从小模块到实板的验证	22
9.1 XSim 独立模块与边界精测	22
9.2 Verilator 全 SoC 大整合	23
9.3 访存和退休结果逐项核对	23
10 CoreMark 性能与 IPC	24
10.1 性能提升策略	24
10.2 Verilator 五次重复性	25
10.3 实板加速 CoreMark	26
11 150 MHz 时序优化与最终布线	27
11.1 每个阶段的建立与保持时间	27
11.2 时序收敛思路	28
11.3 最终最慢路径	28
11.4 资源使用	29
12 最终结果、复现步骤与已知问题	29
12.1 最终实现结果	29
12.2 复现步骤	29
12.3 已知问题与后续工作	30
附录 A 源码文件说明	30
附录 B 工具和版本	30
参考文献	30

摘要

本工程在 Kintex-7 XC7K325T 上实现了一颗双发射、乱序执行、按序精确提交的 RV32 处理器 SoC。核心支持 RV32I、M、Zba、Zicsr 和 Zifencei，集成 64 KiB 统一 BRAM、独立 I/D Cache、UART、定时器和 IRQ/LED MMIO，并移植 RT-Thread Nano 3.1.5 与 FinSH/MSH。官方 PLL 输出 150 MHz 系统时钟和固定 50 MHz 测温显示时钟。测温显示电路具有独立的复位、状态机和数据通路，不接入 CPU 总线、软件状态或 MMIO，因此 CoreMark、RTOS 调度和串口输出不会中断测温与数码管扫描。

性能方面，工程让取指、发射、执行和访存尽量同时工作，并用 Custom-0 指令加速 CoreMark 中的 CRC 和状态计算。时序检查只使用生成该 bit 文件的最终布线 DCP，并逐阶段检查 setup 和 hold。最终结果为 WNS +0.001 ns、TNS 0、WHS +0.045 ns、THS 0。

验证结果

官方 CoreMark 核心算法文件与发布包逐字一致。自研 CPU 用 Custom-0 指令完成结果相同的硬件计算。实板固定运行 15,000 iterations，两轮 CRC 正确且运行时间均超过 10 s。Verilator 的短测试用于检查 RTL 和比较性能。当前检查会把访存结果和退休结果与预先保存的正确副本逐项比较；外部 ISS 全指令逐条比较尚未接入 [23]。

1 工程内容、功能范围与主要成果

1.1 工程目录与生成文件

项目	内容
工程目录	jyd2025_ooo_soc_v23/
bitstream	jyd_soc_top_rththread_coremark_150mhz_wns_p0001_20260815.bit
bit SHA-256	B2F631002F39C4F477AAD68B06DA53017386ED6E9C6DC4051996A741A8809CD3
final DCP	jyd_soc_top_final_route_wns_p0001_20260815.dcp
DCP SHA-256	FB7C6E9BD75E175B4E6D59FE3B043EF951474DB99761F30A7A1BF941650F414B
源码快照	jyd2025_m0097_source_snapshot_20260815_1217.zip

1.2 功能实现范围

特性	工程实现	边界说明
150/50 MHz 双时钟	SoC 与测温/SEG 使用独立状态机、复位链和数据通路	两路时钟来自同一 PLL，电路隔离由顶层连接关系保证

特性	工程实现	边界说明
乱序双发射	2-wide rename/dispatch, 双 8 项 IQ, EX0/EX1 不对称分工, 最多双提交	双发受指令类型、相关性和执行端口约束
串口退格编辑	移植并保留 FinSH/MSH 的 BS/DEL 就地编辑、光标重绘与回车执行	编辑算法来自 RT-Thread 官方 FinSH; 工程贡献是移植、BSP 与实板验证
硬件加速	Custom-0 完成 CRC16、CRC32-word 和状态转换, 链接时用硬件版本替换软件版本	编译参数中写明 -DJYD_COREMARK_HW_ACCEL=1; CRC 保持一致
逐项对照检查	退休记录 + 请求/响应结果副本 + 退休结果副本	未实现外部 ISS 全 ISA 逐指令对比

2 SoC 总体结构与两路时钟

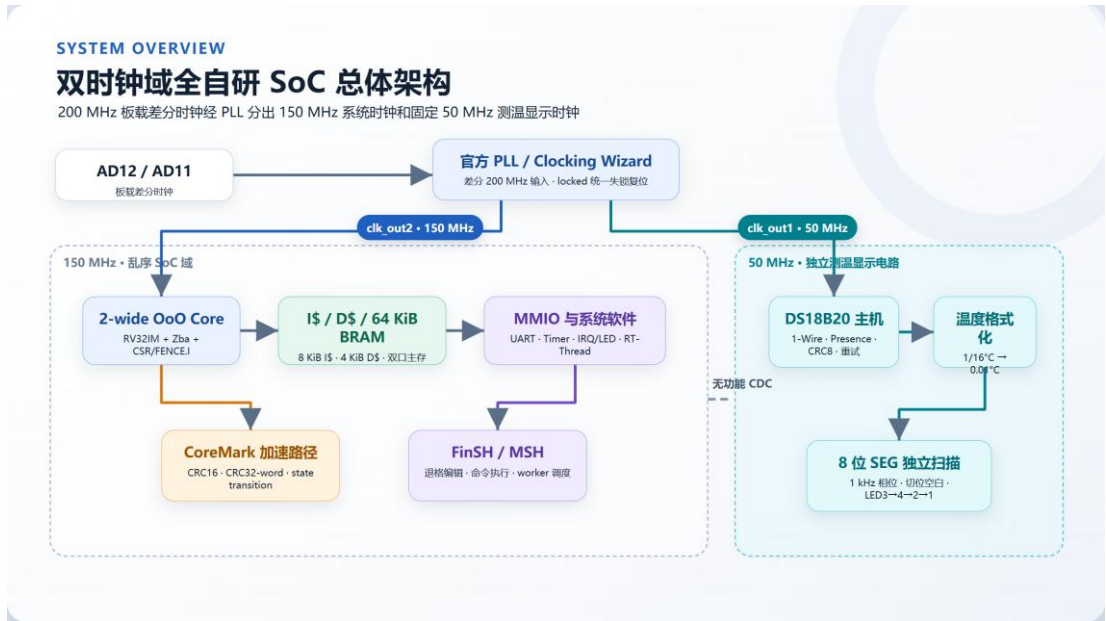


图 1 双时钟域全自研 SoC 总体架构

2.1 时钟与复位

板载 200 MHz 差分时钟从 AD12/AD11 进入 Clocking Wizard。工程使用官方 PLL/XCI 文件和板级约束；Vivado 2025.2 建工程脚本在项目副本中把 clk_out2 设为 150 MHz，clk_out1 保持 50 MHz。PLL 失锁异步置位复位，两个时钟域各自通过两级同步链释放复位 [8][11]。

源码摘录 1：PLL 双输出与双域复位 (rtl/soc/jyd_soc_top.sv:36-52)

```

36: // The only reused IP block is the official active Clocking Wizard XCI.
37: pll u_pll (
38:   .clk_in1_p(i_sys_clk_p),
39:   .clk_in1_n(i_sys_clk_n),
40:   .clk_out1(clk_board_50m),
41:   .clk_out2(clk_soc),
42:   .locked pll_locked)
43: );
44:
45: // PLL lock loss asserts reset asynchronously. Each used clock domain owns

```

```

46: // its own two-flop synchronous deassertion chain.
47: reset_sync u_soc_reset (
48:   .clk_i(clk_soc), .arst_ni(pll_locked), .rst_no(rst_n)
49: );
50:
51: reset_sync u_board_reset (
52:   .clk_i(clk_board_50m), .arst_ni(pll_locked), .rst_no(board_rst_n)

```

2.2 150 MHz 系统与 50 MHz 测温显示电路的隔离

150 MHz 电路包含 CPU、Cache、BRAM、MMIO、UART 和 RT-Thread。50 MHz 电路包含 DQ 开漏驱动、DS18B20 协议状态机、CRC8、温度格式化和 40 根 SEG 管脚。顶层没有把 CPU bus、MMIO、IRQ、软件变量或 clk_soc 信号接入测温显示电路。系统高负载、缓存缺失和 UART 连续输出只影响 150 MHz 系统，不会改变测温周期和数码管扫描周期。

源码摘录 2：50 MHz 测温显示电路的顶层连接（rtl/soc/jyd_soc_top.sv:84-95）

```

84: // synchronizer, the sensor pin, and all SEG pins. No CPU bus, software state,
85: // or clk_soc signal enters this island, so system load cannot stall sampling
86: // or display refresh and there is no inter-domain data CDC to constrain.
87: temperature_display_island #(
88:   .CLK_HZ(50_000_000)
89: ) u_temperature_island (
90:   .clk_i(clk_board_50m),
91:   .rst_ni(board_run_q),
92:   .dq_i(ds18b20_dq_in),
93:   .dq_drive_low_o(ds18b20_drive_low),
94:   .seg_o(virtual_seg)
95: );

```

3 乱序双发射处理器结构

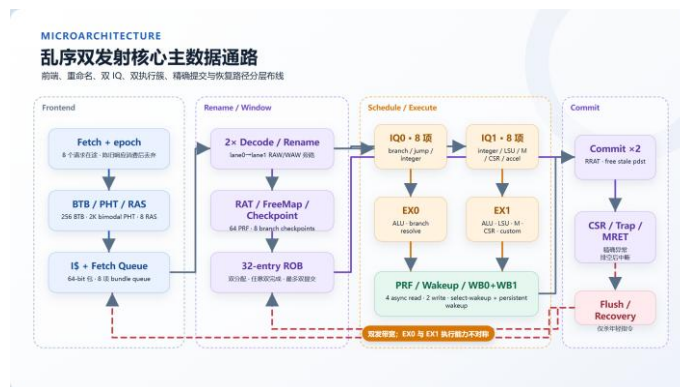


图 2 双发射乱序核心主数据通路

3.1 结构参数

结构	当前参数	说明
取指/解码/重命名	最多 2 条/周期	64-bit IS 响应拆为两条 32-bit 指令，不支持 RVC
ROB（按顺序收尾队列）	32 项，最多双分配/双完成/双提交	偶/奇 bank；uop ID 带代际防止陈旧响应误写
PRF（物理寄存器）	64 个物理寄存器，4 异步读/2 写	按 WB 来源和 lane 复制 LUTRAM，latest bank 选择
IQ0/IQ1（待执行队列）	各 8 项	oldest-ready；弹性整数运算轮转，专用类指令定向路由

结构	当前参数	说明
LQ / SQ (读/写内存队列)	各 8 项	更旧 store 依赖、逐字节 forwarding、提交后 store drain
IS / DS (指令/数据缓存)	8 KiB / 4 KiB	IS 2-way blocking refill; DS 2-way、2 MSHR、write-back/write-allocate
主存	64 KiB、8192×64	真双口 BRAM, A=IS、B=DS

3.2 乱序执行和按顺序收尾

核心采用与 Tomasulo 思想一致的标签依赖跟踪与就绪指令选择 [13]，但架构可见顺序由 32 项 ROB 与 RRAT 维护。乱序完成只标记 ROB 项目和准备 PRF 写回；寄存器映射释放、store/MMIO 副作用和 CSR 更新只能从 ROB head 按序发生，从而满足精确异常的定义 [14]。

源码摘录 3：双 IQ 容量与不对称路由 (rtl/core/ooo_core.sv:603–607, 682–700)

```

603: // Four rows per payload bank give each lane the architectural eight-entry
604: // window. The previous 16-entry-per-lane experiment doubled the age matrix
605: // and global wakeup fanout without enough CoreMark gain to justify its
606: // 150 MHz routing cost.
607: localparam int unsigned IQ_BANK_ENTRIES = 4;
682: function automatic logic route_requires_iq1(input decoded_instr_t d);
683:     if (d.is_load || d.is_store || d.is_muldiv || d.is_csr ||
684:         d.is_serializing || d.illegal || d.is_accel) return 1'b1;
685:     return 1'b0;
686: endfunction
687:
688: function automatic logic route_requires_iq0(input decoded_instr_t d);
689:     return d.is_branch || d.is_jump;
690: endfunction
691:
692: function automatic logic route_is_flexible(input decoded_instr_t d);
693:     return !route_requires_iq1(d) && !route_requires_iq0(d);
694: endfunction
695:
696: function automatic logic needs_checkpoint(input decoded_instr_t d);
697:     // Direct JALs are redirected by response-time predecode before dispatch,
698:     // so only conditional branches and indirect JALR need recovery snapshots.
699:     return d.is_branch || (d.is_jump && d.uses_rs1);
700: endfunction

```

3.3 退休结果输出

commit_trace_o[2] 在每个退休 lane 上输出 PC、指令、寄存器写回、store 地址/数据/字节使能及异常信息。它一方面为调试提供完整的架构边界，另一方面是 Verilator 退休级存储一致性检查的输入。

源码摘录 4：退休 trace 生成 (rtl/core/ooo_core.sv:2210–2223)

```

2210:     commit_trace_o[lane] = '0;
2211:     commit_trace_o[lane].valid = retire_lane_valid[lane];
2212:     commit_trace_o[lane].pc = rob_head_entry[lane].pc;
2213:     commit_trace_o[lane].inst = rob_head_entry[lane].inst;
2214:     commit_trace_o[lane].rd_wen = rob_head_entry[lane].rd_wen;
2215:     commit_trace_o[lane].rd_addr = rob_head_entry[lane].rd_addr;
2216:     commit_trace_o[lane].rd_data = rob_head_entry[lane].result;
2217:     commit_trace_o[lane].mem_wen = head_meta[lane].is_store;
2218:     commit_trace_o[lane].mem_addr = head_meta[lane].store_addr;
2219:     commit_trace_o[lane].mem_data = head_meta[lane].store_data;
2220:     commit_trace_o[lane].mem_mask = head_meta[lane].store_mask;
2221:     commit_trace_o[lane].exception = rob_head_entry[lane].exception;
2222:     commit_trace_o[lane].cause = rob_head_entry[lane].cause;
2223:     commit_trace_o[lane].tval = rob_head_entry[lane].tval;

```

4 取指、分支预测、寄存器分配与恢复

4.1 取指和旧响应丢弃

前端可保持 8 个 ICache 请求在途，每个请求带请求 PC、预测元数据和 4-bit epoch。错误预测、trap、MRET 或 FENCE.I 改变 epoch 后，旧响应仍会被接收以释放下层握手，但不进入 fetch bundle queue。这个“消费但丢弃”规则避免重定向后的陈旧 IS 响应污染新控制流。

4.2 分支预测

部件	规模	实现细节
BTB	128 sets $\times$ 2 ways = 256 entries	直接/间接跳转目标缓存
PHT	2048 $\times$ 2-bit	索引是 PC[12:2]，当前是 bimodal，不是 gshare [15]
RAS	8 entries	发射态/提交态历史及恢复快照
GHR	11-bit	当前用于状态跟踪/快照，未与 PHT 索引异或
待补定向验证 branch_checkpoints 每 bank 已扩到 7 个物理行，但 predictor snapshot 每 bank 仍只有 4 行并仅取 checkpoint_id[1:0]。在 bank 人口偏斜时，row 4–6 可能与 row 0–2 别名。本文档不将“所有 8 个任意检查点的预测历史无别名”列为已验证能力。		

4.3 寄存器分配和分支恢复记录

RAT/RRAT 各为 32 项，物理寄存器为 64 项。lane1 在同周期可见 lane0 的新映射，因而可正确处理同包 RAW/WAW。free bitmap 按奇偶物理寄存器分 bank，优先一奇一偶双分配。分支检查点保存完整 RAT 映射和 ROB 位置；正确预测解析后释放，错误预测时恢复 RAT，并归还被杀年轻指令的 pdst。

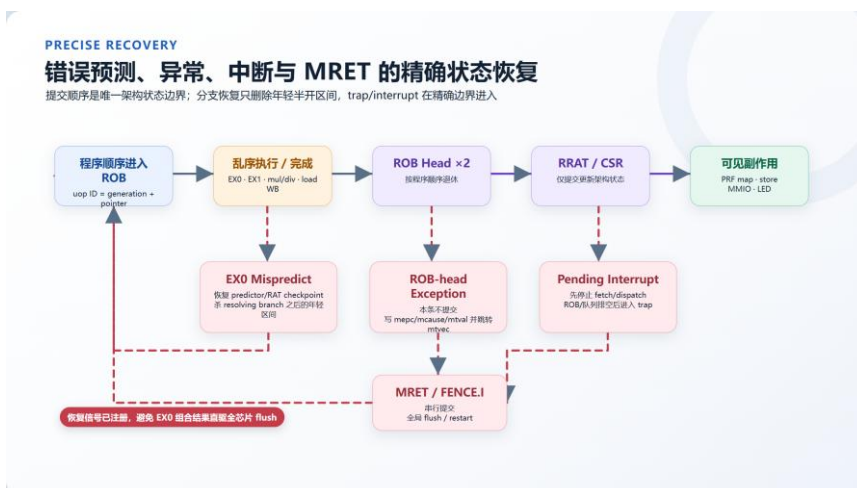


图3 错误预测、异常、中断与 MRET 精确恢复

5 执行单元、读写内存与缓存

5.1 EX0 / EX1 分工

EX0 执行整数 ALU、JAL/JALR 与条件分支，并生成实际方向、目标和错误预测信息。EX1 执行第二整数 ALU、LSU、CSR/串行指令、M 扩展和三条 Custom-0 加速指令。这种“2-wide 但功能不完全对称”的方案在保持双发射机会的同时，避免复制整套 LSU/branch/M/CSR 硬件所带来的局部互连膨胀。这与超标量处理器中 wakeup/select 和 bypass 复杂度可主导周期时间的观察一致 [17]。

5.2 读写内存接口与请求队列

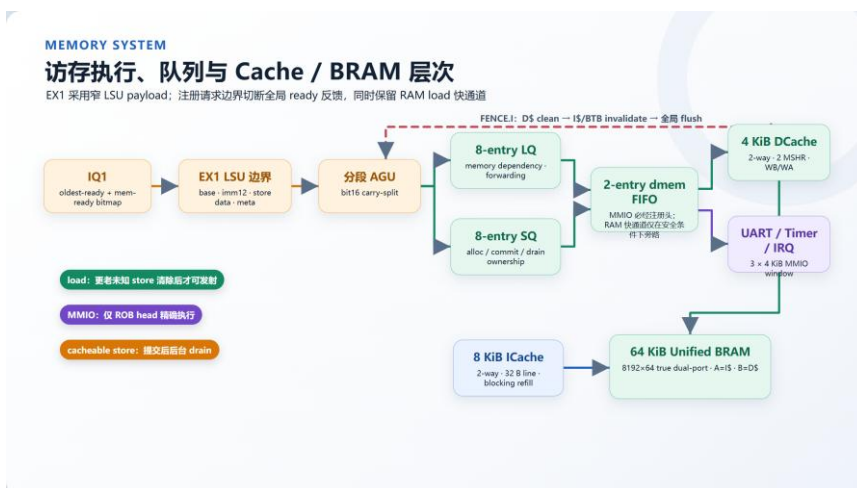


图4 访存执行、队列与 Cache/BRAM 层次

EX1 只将 base、imm12、store data 和紧凑元数据带过寄存器边界；地址形成、对齐判断、LQ/SQ 依赖和请求格式化在边界之后完成。dmemo_request_fifo 深度为 2，MMIO 必须首先进入注册头，从而终止 peripheral-ready → decode/ROB 的全局组合反馈。RAM load 只在 FIFO 为空且来自 EX-local fast lane 时保留 fall-through，而 LQ 扫描/重放 load 先进注册头，把宽 oldest-selector 从 DCache tag 路径移开。

源码摘录 5: dmem FIFO 与受控 fall-through (rtl/soc/dmem_request_fifo.sv:64-74)

```
64: assign enq_ready_o = count_q < COUNT_W'(DEPTH);
65: // Cacheable stores and the EX-local load lane retain zero-added-latency
66: // fall-through. A replay/scanned load first enters the registered head;
67: // this keeps its wide LQ selector out of the D-cache address/tag cone.
68: // MMIO and unmapped requests first enter the FIFO, terminating their decode
69: // and peripheral-ready path at this local boundary.
70: assign fallthrough = (count_q == 0) &&
71: soc_cfg_pkg::addr_is_ram(enq_addr_i) &&
72: (enq_write_i || enq_fast_load_i);
73: assign deq_valid_o = (count_q != 0) ||
74: (fallthrough && enq_valid_i && enq_ready_o);
```

5.3 指令缓存、数据缓存与主存

组件	组织	行为
ICache	128 sets、2-way、32 B/line、4×64-bit beat	每周期中响应；单 miss blocking refill；FENCE.I 逐 set 无效
DCache	64 sets、2-way、32 B/line、2 MSHR	write-back/write-allocate；同 line miss merge；hit-under-miss；store overlay
Unified BRAM	64 KiB, 8192×64 true dual-port	Port A 专属 ICache, Port B 专属 DCache

6 指令、系统寄存器、异常与硬件加速

6.1 已实现的指令

指令集	实现范围	备注
RV32I	整数计算、Load/Store、branch、JAL/JALR、FENCE、ECALL/EBREAK	32-bit XLEN, x0 恒为 0 [1]
M	MUL/MULH/MULHSU/MULHU、DIV/DIVU/REM/REMU	乘法 4 个 16×16 DSP 部分积；除法 32 步迭代 [1]
Zba	SH1ADD/SH2ADD/SH3ADD	地址生成扩展 [1]
Zicsr	CSRRW/CSRRS/CSRRC 及 immediate 形式	M-mode CSR [1][2]
Zifencei	FENCE.I	等待 D\$ clean, 再失效 I\$/BTB 并全局重启 [1]
JYD Custom-0	CRC16、CRC32-word、state transition	opcode 0x0B, funct3=0/2/1

未实现 RVC、A、F/D 和 S/U 特权态。编译目标是 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32。规范引用说明应有指令语义，本工程的实现正确性仍由 XSim、Verilator、CRC 和实板证据支撑。

6.2 系统寄存器与中断

支持 mstatus/misa/mie/mtvec/mscratch/mepc/mcause/mtval/mip、mcycle[h]/minstret[h]、

cycle[h]/instret[h] 和固定 ID CSR。mtvec 为 direct mode，中断优先级 external > timer > software（software IRQ=0）。pending 时停止 fetch/dispatch；take_interrupt 仅以 ROB count=0 为精确边界，未退休操作先完成，已提交 store 可由 SQ 继续排空。memory_queues_empty 只用于 MRET、FENCE.I 等恢复路径；随后写 mepc/mcause 并转入 mtvec [2][14]。

6.3 CoreMark 自定义加速

官方 CoreMark 核心算法文件文本与发布包一致。software/build_firmware.ps1 在编译后定向弱化 crc16、cru16、cru32 和 core_state_transition 对象符号，coremark_accel.c 的强符号 overlay 以 .insn 发射 Custom-0。CRC16 为 16 步反馈展开，CRC32-word 为 GF(2) XOR 矩阵，state transition 返回下一状态与索引。XSim 覆盖 state 的 8×256 组合、CRC16 的 4 项定向+1000 项随机、CRC32-word 的 1 项已知+1024 项随机；5 次 Verilator 和 2 次实板均保持预期 CRC [5][23]。

7 RT-Thread Nano 3.1.5 移植实现



图 5 RT-Thread 从复位入口到线程调度的实现路径

7.1 移植范围与源码组成

工程直接纳入 RT-Thread v3.1.5 的内核、RISC-V E310 CPU 端口和 FinSH/MSH 源码，不使用板级厂商 BSP。自研部分负责复位入口、64 KiB 链接布局、UART/定时器寄存器访问、trap 分发、线程栈对齐、MSH 应用命令和 CoreMark worker。Nano 配置不启用动态堆，main、idle、tshell 和 CoreMark 线程的控制块与栈均由静态数组提供 [3][4]。

层次	纳入文件	本工程连接点
RT-Thread 内核	src/{clock,components,idle,ipc,irq,kservice,object,scheduler,thread,timer}.c	100 Hz tick、32 级优先级、静态线程、信号量

层次	纳入文件	本工程连接点
RISC-V CPU 端口	libcpu/risc-v/e310/{context_gcc.S,entry_gcc.S,stack.c}	RV32 寄存器现场、mstatus/mepc、线程切换与中断返回
FinSH/MSH	components/finsh/{shell,msh,cmd}.c	官方行编辑、命令查找、tshell 线程
自研 BSP/应用	software/rtthread/{start.S,board.c,app.c,msh_demo.c,linker.ld}	启动、MMIO、trap、CoreMark 与演示命令

源码摘录 6：RT-Thread 与 RISC-V 端口的编译输入（software/build_firmware.ps1:143–164）

```

143:     $RtSources = @(
144:         (Join-Path $RtBsp 'start.S'),
145:         (Join-Path $RtBsp 'board.c'),
146:         (Join-Path $RtBsp 'app.c'),
147:         (Join-Path $SoftwareRoot 'common\platform.c'),
148:         (Join-Path $RtRoot 'src\clock.c'),
149:         (Join-Path $RtRoot 'src\components.c'),
150:         (Join-Path $RtRoot 'src\idle.c'),
151:         (Join-Path $RtRoot 'src\ipc.c'),
152:         (Join-Path $RtRoot 'src\irq.c'),
153:         (Join-Path $RtRoot 'src\kservice.c'),
154:         (Join-Path $RtRoot 'src\object.c'),
155:         (Join-Path $RtRoot 'src\scheduler.c'),
156:         (Join-Path $RtRoot 'src\thread.c'),
157:         (Join-Path $RtRoot 'src\timer.c'),
158:         (Join-Path $Finsh 'shell.c'),
159:         (Join-Path $Finsh 'msh.c'),
160:         (Join-Path $Finsh 'cmd.c'),
161:         (Join-Path $MiniLibc 'string.c'),
162:         (Join-Path $RtRoot 'libcpu\risc-v\e310\context_gcc.S'),
163:         (Join-Path $RtRoot 'libcpu\risc-v\e310\entry_gcc.S'),
164:         (Join-Path $RtRoot 'libcpu\risc-v\e310\stack.c')

```

7.2 复位入口与 64 KiB 内存布局

CPU 从 0x0000_0000 的 `_start` 开始执行。入口先关闭 mie 和 `mstatus.MIE`，建立 `gp` 与 16 字节对齐的启动栈，逐字清零 `.bss`，把 `mtvec` 写为 `trap_entry`，最后调用 `rtthread_startup`。若启动函数异常返回，软件向 0x0000_FFF0 写失败签名并停机，仿真可以直接识别该状态。

源码摘录 7：复位入口、BSS 清零与 `mtvec`（software/rtthread/start.S:6–26）

```

6: _start:
7:     csrwi    mie, 0
8:     csrci    mstatus, 8
9:
10:    .option push
11:    .option norelax
12:    la        gp, __global_pointer$
13:    .option pop
14:    la        sp, __boot_stack_top
15:
16:    la        t0, _bss_start
17:    la        t1, _bss_end
18: 1:
19:    bgeu      t0, t1, 2f
20:    sw        zero, 0(t0)
21:    addi      t0, t0, 4
22:    j         1b
23: 2:
24:    la        t0, trap_entry
25:    csrw      mtvec, t0
26:    call      rtthread_startup

```

区域	当前地址	作用
.text/.rodata/.data/.bss	0x0000_0000–0x0000_EABF	代码、常量、初始化数据、内核对象和线程栈； _bss_end=0xEAC0
未分配余量	0x0000_EAC0–0x0000_EF5F	不启用 heap；链接断言保证映像不越过保留栈
boot stack	0x0000_EFA0–0x0000_F79F	启动阶段和非线程代码使用，2 KiB
IRQ stack	0x0000_F7E0–0x0000_FDF	trap C 处理函数独立使用，2 KiB
完成签名	0x0000_FFF0–0x0000_FFFF	仿真 PASS/FAIL 与异常状态

当前 `rtthread_board.bin` 为 43,660 B；链接后的 `.bss` 末端、两块 2 KiB 保留栈和 64 B guard 均

由 linker.ld 固定，并用 ASSERT 检查对齐与重叠。

源码摘录 8：启动栈、IRQ 栈与越界断言（software/rtthread/linker.ld:58-73）

```
58: /* Top-of-RAM ownership: signature, IRQ stack, guard, boot stack, guard. */
59: _signature_addr = ORIGIN(RAM) + LENGTH(RAM) - 16;
60: __irq_stack_top = _signature_addr - 16;
61: __irq_stack_bottom = __irq_stack_top - 2048;
62: __irq_guard_bottom = __irq_stack_bottom - 64;
63: __boot_stack_top = __irq_guard_bottom;
64: __boot_stack_bottom = __boot_stack_top - 2048;
65: __boot_guard_bottom = __boot_stack_bottom - 64;
66: __heap_end = __boot_guard_bottom;
67: _sp = __irq_stack_top;
68:
69: /DISCARD/ : { *(.comment) *(.note*) *(.eh_frame*) }
70:
71: ASSERT((__sp & 15) == 0, "IRQ stack is not 16-byte aligned")
72: ASSERT((__boot_stack_top & 15) == 0, "boot stack is not 16-byte aligned")
73: ASSERT(__bss_end < __heap_end, "RT-Thread image overlaps reserved stacks")
```

7.3 内核启动顺序与线程建立

rtthread_startup 先保持全局中断关闭，调用 rt_hw_board_init 配置 UART、mtvec、MTIE 和周期定时器；随后初始化软件定时器、调度器、main 线程和 idle 线程。rt_system_scheduler_start 从 32 位 ready bitmap 找到最高优先级线程，把 rt_current_thread 指向该线程，再通过 rt_hw_context_switch_to 恢复首个线程现场。main 线程中的 rt_components_init 执行 INIT_APP_EXPORT 项，FinSH 因而在 main() 之前建立 tshell。

源码摘录 9：RT-Thread 启动次序（software/third_party/rt-thread-3.1.5/src/components.c:212-245）

```
212: int rtthread_startup(void)
213: {
214:     rt_hw_interrupt_disable();
215:
216:     /* board level initialization
217:      * NOTE: please initialize heap inside board initialization.
218:      */
219:     rt_hw_board_init();
220:
221:     /* show RT-Thread version */
222:     rt_show_version();
223:
224:     /* timer system initialization */
225:     rt_system_timer_init();
226:
227:     /* scheduler system initialization */
228:     rt_system_scheduler_init();
229:
230: #ifdef RT_USING_SIGNALS
231:     /* signal system initialization */
232:     rt_system_signal_init();
233: #endif
234:
235:     /* create init_thread */
236:     rt_application_init();
237:
238:     /* timer thread initialization */
239:     rt_system_timer_thread_init();
240:
241:     /* idle thread initialization */
242:     rt_thread_idle_init();
243:
244:     /* start scheduler */
245:     rt_system_scheduler_start();
```

线程	优先级	时间片	栈	建立位置与职责
coremark	5	10 tick	8192 B	app.c 静态建立；等待 cmsem，执行 CoreMark
main	10	20 tick	2048 B	components.c 静态建立；运行组件初始化和应用 main
tshell	20	10 tick	2048 B	finsh_system_init 静态建立；接收、编辑并执行 MSH 命令
tidle	31	—	512 B	内核建立；没有其他 ready 线程时运行

7.4 线程栈与上下文切换

每个新线程先生成 32 个 32 位字的初始现场。epc 指向线程入口，ra 指向线程退出函数，a0 保存入口参数；mstatus 写入 MPP=Machine、MPIE=1，使第一次 mret 进入线程时允许后续中断。SP 在建帧前向下对齐到 16 字节，满足当前 Clang RV32 ilp32 ABI。gp 在复位入口一次建立并保持全局不变，其余通用寄存器、mepc 和中断使能状态由汇编保存。

源码摘录 10：线程初始现场和 16 字节栈对齐（software/third_party/rt-thread-3.1.5/libcpu/risc-v/e310/stack.c:64–90）

```
64: rt_uint8_t *rt_hw_stack_init(void *tentry, void *parameter,
65:                               rt_uint8_t *stack_addr, void *texit)
66: {
67:     struct stack_frame *stack_frame;
68:     rt_uint8_t *stk;
69:     int i;
70:
71:     stk = stack_addr + sizeof(rt_uint32_t);
72:     /* The JYD RV32IM ABI contract requires every thread SP to be 16-byte aligned. */
73:     stk = (rt_uint8_t *)RT_ALIGN_DOWN((rt_uint32_t)stk, 16);
74:     stk -= sizeof(struct stack_frame);
75:
76:     stack_frame = (struct stack_frame *)stk;
77:
78:     for (i = 0; i < sizeof(struct stack_frame) / sizeof(rt_ubase_t); i++)
79:     {
80:         ((rt_ubase_t *)stack_frame)[i] = 0xdeadbeef;
81:     }
82:
83:     stack_frame->ra = (rt_ubase_t)texit;
84:     stack_frame->a0 = (rt_ubase_t)parameter;
85:     stack_frame->epc = (rt_ubase_t)tentry;
86:
87:     // force to machine mode(MPP=11) and set MPIE to 1
88:     stack_frame->mstatus = 0x00001880;
89:
90:     return stk;
```

线程主动切换时，rt_hw_context_switch 把当前 SP 写回 from_thread->sp，恢复 to_thread->sp 后执行 mret。中断中发生调度时，rt_hw_context_switch_interrupt 只记录 from/to 指针并置位切换标志；trap 尾部在所有 C 级中断处理完成后才保存旧 SP、装载新 SP，避免在中断处理过程中覆盖线程现场。

7.5 定时中断与抢占调度

150 MHz 系统时钟和 RT_TICK_PER_SECOND=100 得到 1,500,000 个时钟/节拍。BSP 向 RELOAD 和 COUNT 写 1,499,999，再置 CTRL[1:0]=11，同时启动计数和定时中断。计数到零时硬件重装并置 pending；trap 入口先把被打断线程的通用寄存器压入原线程栈，再切换到专用 IRQ 栈。handle_trap 对 mcause=0x8000_0007 写 ACK、调用 rt_tick_increase，内核据此递减时间片并决定是否切换线程。

源码摘录 11：BSP 定时器配置与 tick 处理（software/rtthread/board.c:53–84）

```
53: static void rt_hw_timer_init(void)
54: {
55:     #ifdef RTTHREAD_TIMER_RELOAD_CYCLES
56:         jyd_u32 reload = RTTHREAD_TIMER_RELOAD_CYCLES - 1u;
57:     #else
58:         jyd_u32 reload = (jyd_clock_hz() / RT_TICK_PER_SECOND) - 1u;
59:     #endif
60:
61:     JYD_TIMER_CTRL = 0u;
62:     JYD_TIMER_ACK = 1u;
63:     JYD_TIMER_RELOAD = reload;
64:     JYD_TIMER_COUNT = reload;
65:     JYD_TIMER_CTRL = 3u;
66: }
67:
68: void rt_hw_board_init(void)
69: {
70:     uart_init(115200u);
```

```

71:     rt_hw_interrupt_init();
72:     rt_hw_timer_init();
73: }
74:
75: void handle_trap(jyd_u32 mcause, jyd_u32 epc, void *irq_stack)
76: {
77:     jyd_u32 mtval;
78:     (void)irq_stack;
79:
80:     if (mcause == MCAUSE_MACHINE_TIMER) {
81:         JYD_TIMER_ACK = 1u;
82:         rt_tick_increase();
83:         return;
84:     }

```

源码摘录 12: trap 使用独立 IRQ 栈并在尾部换线程 (software/third_party/rt-thread-3.1.5/libcpu/risc-v/e310/entry_gcc.S:55–88)

```

55:     // switch to interrupt stack
56:     move s0, sp
57:     la    sp, _sp
58:
59:     // interrupt handle
60:     call rt_interrupt_enter
61:     csrr a0, mcause
62:     csrr a1, mepc
63:     mv a2, sp
64:     call handle_trap
65:     call rt_interrupt_leave
66:
67:     // switch to from thread stack
68:     move sp, s0
69:
70:     // need to switch new thread
71:     la    s0, rt_thread_switch_interrupt_flag
72:     lw    s2, 0(s0)
73:     beqz s2, spurious_interrupt
74:     sw    zero, 0(s0)
75:
76:     csrr a0, mepc
77:     sw a0, 0 * 4(sp)
78:
79:     la    s0, rt_interrupt_from_thread
80:     lw    s1, 0(s0)
81:     sw    sp, 0(s1)
82:
83:     la    s0, rt_interrupt_to_thread
84:     lw    s1, 0(s0)
85:     lw    sp, 0(s1)
86:
87:     lw    a0, 0 * 4(sp)
88:     csrw mepc, a0

```

entry_gcc.S 在使用 t0 生成 MPIE 保存值之前先保存被打断的 x5，返回时再恢复 x5；这项修正确保周期中断不会把普通线程的临时寄存器改成 0x80。同步异常会输出 mcause、mepc、mtval，并写失败签名后停机，实板与仿真都能留下明确故障位置。

7.6 FinSH/MSH 注册、输入与退格编辑

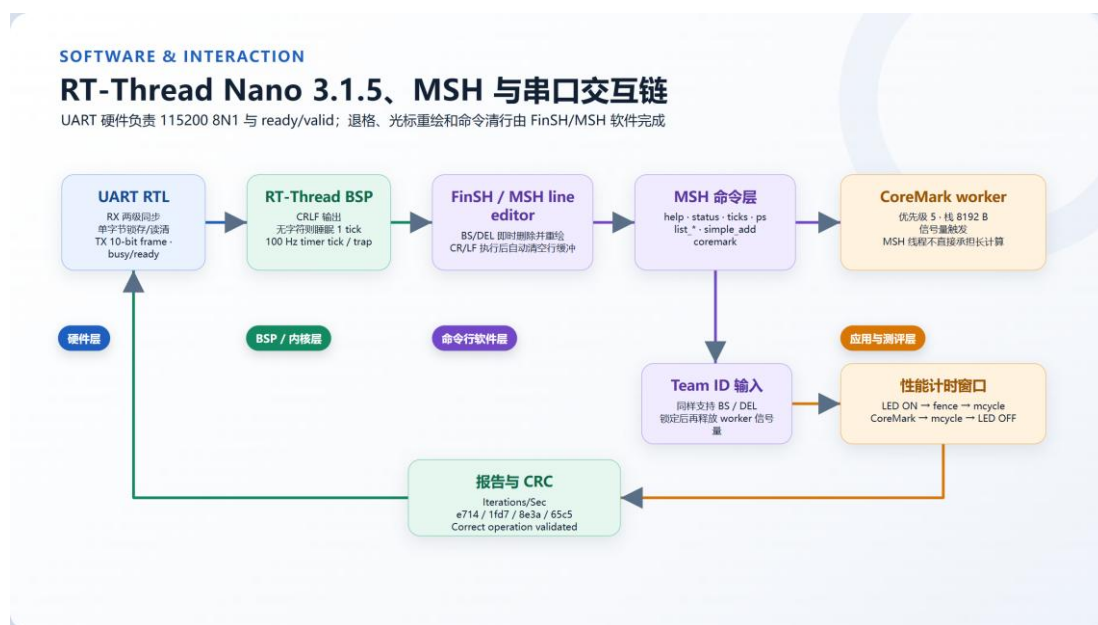


图 6 UART、BSP、FinSH/MSH 与 CoreMark worker 的交互链

MSH_CMD_EXPORT 把命令描述放入 FSymTab。链接脚本用 KEEP 保留 FSymTab/VSymTab 并导出起止符号，finsh_system_init 扫描该区建立命令表；INIT_APP_EXPORT 又把初始化函数放入 .rti_fn，main 线程执行组件初始化时自动建立 tshell。整个注册过程不依赖动态装载。

源码摘录 13：MSH 命令表与组件初始化段（software/rtthread/linker.ld:25-38）

```

25: .rti_fn : ALIGN(4)
26: {
27:     KEEP(*(SORT(.rti_fn*)))
28: } > RAM
29:
30: .finsh_symtab : ALIGN(4)
31: {
32:     __fsymtab_start = .;
33:     KEEP(*(FSymTab))
34:     __fsymtab_end = .;
35:     __vsymtab_start = .;
36:     KEEP(*(VSymTab))
37:     __vsymtab_end = .;
38: } > RAM
  
```

UART 只有 1 字节 RX 锁存，因此 rt_hw_console_getchar 在没有字符时轮询，避免按 10 ms tick 睡眠造成 115200 baud 连续输入丢字；更高优先级线程仍可由定时中断抢占 tshell。FinSH 在软件层识别 BS(0x08) 和 DEL(0x7F)，立即删除光标前字符并重绘后续内容；回车只负责执行已经编辑好的命令，用户不需要先回车清空错误输入。

源码摘录 14：FinSH 的 BS/DEL 就地编辑（software/third_party/rt-thread-3.1.5/components/finsh/shell.c:657-688）

```

657: /* handle backspace key */
658: else if (ch == 0x7f || ch == 0x08)
659: {
660:     /* note that shell->line_curpos >= 0 */
661:     if (shell->line_curpos == 0)
662:         continue;
663:
664:     shell->line_position--;
665:     shell->line_curpos--;
666:
667:     if (shell->line_position > shell->line_curpos)
668:     {
  
```

```

669:         int i;
670:
671:         rt_memmove(&shell->line[shell->line_curpos],
672:                   &shell->line[shell->line_curpos + 1],
673:                   shell->line_position - shell->line_curpos);
674:         shell->line[shell->line_position] = 0;
675:
676:         rt_kprintf("\b%s \b", &shell->line[shell->line_curpos]);
677:
678:         /* move the cursor to the origin position */
679:         for (i = shell->line_curpos; i <= shell->line_position; i++)
680:             rt_kprintf("\b");
681:     }
682:     else
683:     {
684:         rt_kprintf("\b \b");
685:         shell->line[shell->line_position] = 0;
686:     }
687:
688:     continue;

```

7.7 MSH 命令与 CoreMark 工作线程

命令	执行内容
help	遍历 FSymTab, 列出命令名和说明
status	显示 RT-Thread 版本、150 MHz、tick、CoreMark busy/runs
ticks	读取 rt_tick_get
coremark [team_id]	输入或直接给出 Team ID, 释放 cmsem, 由优先级 5 worker 执行
ps / list_thread	显示线程状态、优先级、SP、栈容量和使用率
version	输出 RT-Thread 3.1.5 版本信息
list_sem / list_timer	显示 cmsem、shrx 和系统定时器对象
simple_add	运行 1 到 99/199...999 的求和演示

coremark 命令本身不在 tshell 中执行长测试。命令只检查 busy、收集 Team ID、置 busy 并释放 cmsem; coremark worker 醒来后调用 coremark_entry, 结束时递增 runs、清 busy 并回到永久等待。这样 MSH 线程栈与 CoreMark 的 8192 B 工作栈彼此独立, 重复执行不会复用上一次的运行现场。

源码摘录 15: CoreMark worker 与信号量 (software/rtthread/app.c:39-64)

```

39: static void queue_coremark_with_team_id(const char *team_id)
40: {
41:     set_coremark_team_id(team_id);
42:     coremark_busy = 1u;
43:     rt_kprintf("Team ID locked: %s\n", coremark_team_id);
44:     rt_kprintf("Starting CoreMark, please wait...\n");
45:     rt_kprintf("=====\n");
46:     rt_sem_release(&coremark_sem);
47: }
48:
49: static void coremark_worker(void *parameter)
50: {
51:     int result;
52:
53:     (void)parameter;
54:     for (;;) {
55:         rt_sem_take(&coremark_sem, RT_WAITING_FOREVER);
56:         rt_kprintf("\nCoreMark Team ID: %s\n", coremark_team_id);
57:         rt_kprintf("COREMARK BEGIN (RT-Thread worker, stack=%u)\n",
58:                   (unsigned int)sizeof(coremark_stack));
59:         result = coremark_entry();
60:         coremark_runs++;
61:         coremark_busy = 0u;
62:         rt_kprintf("CoreMark Team ID: %s\n", coremark_team_id);
63:         rt_kprintf("COREMARK DONE rc=%d runs=%u\n", result,
64:                   (unsigned int)coremark_runs);

```

源码摘录 16: LED 只覆盖性能计时窗口 (software/coremark_port/core_portme.c:29-45)

```
29: void start_time(void)
30: {
31:     JYD_COREMARK_LED = 1u;
32:     __asm__ volatile("fence iorw, iorw" ::: "memory");
33:     start_time_value = read_mcycle_shifted8();
34: }
35:
36: void stop_time(void)
37: {
38:     stop_time_value = read_mcycle_shifted8();
39:     JYD_COREMARK_LED = 0u;
40:     __asm__ volatile("fence iorw, iorw" ::: "memory");
41: }
42:
43: CORE_TICKS get_time(void)
44: {
45:     return stop_time_value - start_time_value;
```

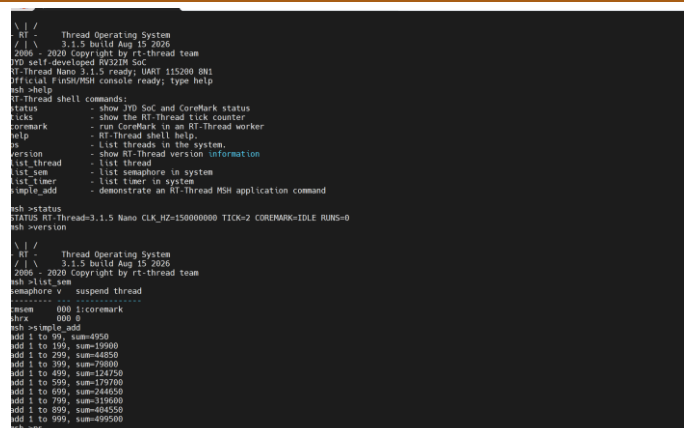


图 7 RT-Thread Nano 3.1.5 启动、MSH 帮助、状态与对象命令

7.8 固件编译与 BRAM 初始化

build_firmware.ps1 对每个源文件独立调用 Clang，只有 CoreMark 的 core_main.c 接收 main=coremark_entry，避免把 RT-Thread 应用 main 一并改名。目标选项为 rv32im_zba_zicr_zifencei/ilp32、-O3、freestanding、无宿主库和无栈保护；链接使用 lld、linker.ld、--gc-sections 和无 build-id。rtthread_board 固定 ITTERS=15000，链接后由 llvmoobjcopy 生成二进制，再由 bin_to_mem64.py 按 64 位字转换为 BRAM 初始化文件。

源码摘录 17: 编译目标与优化选项 (software/build_firmware.ps1:34-49)

```
34: $Common = @(
35:     '--target=riscv32-unknown-elf',
36:     '-march=rv32im_zba_zicr_zifencei',
37:     '-mabi=ilp32',
38:     '-$OptimizationLevel',
39:     '-ffreestanding',
40:     '-fno-builtin',
41:     '-fno-common',
42:     '-fno-stack-protector',
43:     '-msmall-data-limit=0',
44:     '-ffunction-sections',
45:     '-fdata-sections',
46:     '-Wall',
47:     '-Wextra',
48:     '-Wno-unused-parameter',
49:     '-I', (Join-Path $SoftwareRoot 'common')
```

8 外设功能设计与传感器显示

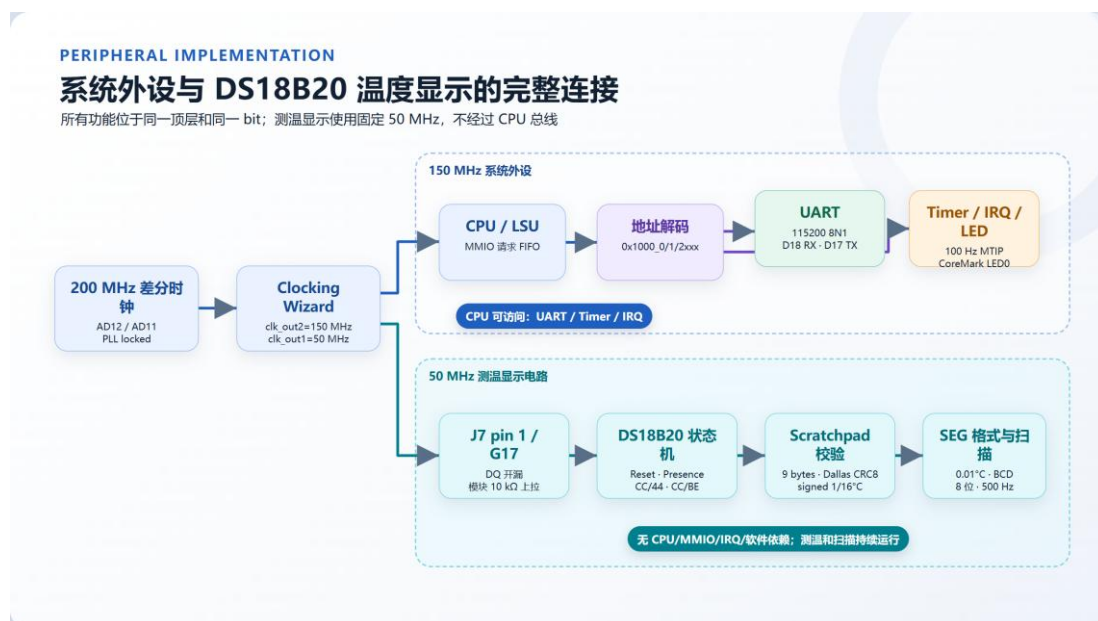


图 8 系统外设与 50 MHz 测温显示电路的完整连接

8.1 地址解码与总线响应

地址范围	模块	寄存器与作用
0x0000_0000-0x0000_FFFF	64 KiB BRAM	统一存放指令、数据、RT-Thread 对象和线程栈
0x1000_0000-0x1000_0FFF	UART	TXDATA、RXDATA、STATUS、CTRL、BAUD_DIV
0x1000_1000-0x1000_1FFF	Tick Timer	COUNT、RELOAD、CTRL、PENDING、ACK
0x1000_2000-0x1000_2FFF	IRQ / Debug	PENDING、ENABLE、ACK、BUILD_ID、CLK_HZ、CoreMark LED

mmio_decode 只检查 0x1000_0xxx、0x1000_1xxx 和 0x1000_2xxx 三个 4 KiB 窗口，并分别只向一个从设备传递 valid。默认响应 ready=1、error=1，命中外设后改用该外设的 ready/rdata/error。请求方向与 ready 返回方向分开组合，避免地址选择和 UART backpressure 形成组合环。DS18B20 与 SEG 不映射到 CPU 地址空间，不占用 MMIO、DCache 或请求 FIFO。

8.2 UART 115200 8N1

偏移	读写	位定义
0x00 TXDATA	写	低 8 位装入 {stop=1,data[7:0],start=0}；发送忙时 ready=0，CPU 保持请求
0x04 RXDATA	读	返回接收字节；成功读取同周期清 RX valid
0x08 STATUS	读	bit0=TX ready, bit1=RX valid
0x0C CTRL	读写	bit0=RX IRQ enable, bit1=TX-ready IRQ enable
0x10 BAUD_DIV	读写	150 MHz 时写 1302；最小值钳位为 2

RX 管脚先经过两级 ASYNC_REG 同步。检测到低电平后等待半个 bit 再确认 start，随后每隔 baud_div 个系统周期在位中心采样 8 个数据位，stop 位为高才发布字节。TX 在寄存器内依次保持 start、8 个 LSB-first 数据位和 stop，每位持续 baud_div 个周期。150 MHz/1302=115,207.37 baud，与 115,200 的误差约 +0.0064%。

源码摘录 18: UART 起始位中点确认和逐位采样 (rtl/periph/uart.sv:124–170)

```

124:     if (bus_fire && !bus_req_i.write &&
125:         (bus_req_i.addr[11:0] == REG_RXDATA)) begin
126:         rx_valid_q <= 1'b0;
127:     end
128:
129:     unique case (rx_state_q)
130:     RX_IDLE: begin
131:         if (!rx_sync_q[1]) begin
132:             rx_state_q <= RX_START;
133:             rx_count_q <= baud_div_q >> 1;
134:         end
135:     end
136:     RX_START: begin
137:         if (rx_count_q == 0) begin
138:             if (!rx_sync_q[1]) begin
139:                 rx_state_q <= RX_DATA;
140:                 rx_count_q <= baud_div_q - 1'b1;
141:                 rx_bit_q <= '0;
142:             end else begin
143:                 rx_state_q <= RX_IDLE;
144:             end
145:         end else begin
146:             rx_count_q <= rx_count_q - 1'b1;
147:         end
148:     end
149:     RX_DATA: begin
150:         if (rx_count_q == 0) begin
151:             rx_shift_q[rx_bit_q] <= rx_sync_q[1];
152:             rx_count_q <= baud_div_q - 1'b1;
153:             if (rx_bit_q == 3'd7) begin
154:                 rx_state_q <= RX_STOP;
155:             end else begin
156:                 rx_bit_q <= rx_bit_q + 1'b1;
157:             end
158:         end else begin
159:             rx_count_q <= rx_count_q - 1'b1;
160:         end
161:     end
162:     RX_STOP: begin
163:         if (rx_count_q == 0) begin
164:             if (rx_sync_q[1]) begin
165:                 rx_data_q <= rx_shift_q;
166:                 rx_valid_q <= 1'b1;
167:             end
168:             rx_state_q <= RX_IDLE;
169:         end else begin
170:             rx_count_q <= rx_count_q - 1'b1;

```

8.3 周期定时器、外部中断与 CoreMark LED

Tick Timer 是 32 位向下计数器。CTRL[0] 使能计数，CTRL[1] 使能 MTIP；COUNT 到零时装回 RELOAD 并置 PENDING，向 ACK 写 1 清 pending。IRQ/Debug 把 UART 中断锁存为 4 位 pending，ENABLE 决定 MEIP，ACK 逐位清除；BUILD_ID=0x2025_2301、CLK_HZ=150000000 供软件核对当前 bit，LED 寄存器 bit0 直接连接 virtual_led[0]。

源码摘录 19: 定时器重装、pending 与 ACK (rtl/periph/tick_timer.sv:40–73)

```

40:     always_ff @(posedge clk_i) begin
41:         if (!rst_ni) begin
42:             count_q <= '0;
43:             reload_q <= '0;
44:             enable_q <= 1'b0;
45:             irq_enable_q <= 1'b0;
46:             pending_q <= 1'b0;
47:         end else begin
48:             if (enable_q) begin
49:                 if (count_q == 0) begin
50:                     count_q <= reload_q;
51:                     pending_q <= 1'b1;
52:                 end else begin
53:                     count_q <= count_q - 1'b1;
54:                 end

```

```

55:     end
56:
57:     if (bus_fire && bus_req_i.write) begin
58:         unique case (bus_req_i.addr[11:0])
59:             REG_COUNT: begin
60:                 if (!bus_req_i.wstrb) count_q <= bus_req_i.wdata;
61:             end
62:             REG_RELOAD: begin
63:                 if (!bus_req_i.wstrb) reload_q <= bus_req_i.wdata;
64:             end
65:             REG_CTRL: begin
66:                 if (bus_req_i.wstrb[0]) begin
67:                     enable_q <= bus_req_i.wdata[0];
68:                     irq_enable_q <= bus_req_i.wdata[1];
69:                 end
70:             end
71:             REG_ACK: begin
72:                 if (bus_req_i.wdata[0]) pending_q <= 1'b0;
73:             end

```

CoreMark port 在开始计时前写 LED=1 并执行 fence，在读取结束周期后写 LED=0。LED 因而只覆盖被计时的算法区间，不包含 Team ID 输入、线程唤醒和结果打印；其余 31 个 LED 固定为 0。

8.4 DS18B20 电气连接与管脚约束

连接	板上位置	电气约束
DQ / DAT	J7 pin 1, DEBUG_1, FPGA G17	LVC MOS33, DRIVE 4, SLEW SLOW, 开漏驱动
VCC	J7 pin 20, 3.3 V	传感器外部供电；不经过 FPGA I/O
GND	J7 pin 19	板地
上拉	传感器端子模块自带 10 kΩ	XDC 不再启用内部 PULLUP

顶层只允许 ds18b20_drive_low=1 时把 DQ 拉到 0；其余时间输出高阻，由模块上的 10 kΩ 电阻拉到 3.3 V，FPGA 从同一双向管脚读取电平。DQ 输入仅把外部管脚到第一级同步触发器的路径设为 false path，第一级到第二级以及后续 50 MHz 状态机仍做完整 STA；DQ 输出是按微秒协议定时的异步板外接口。

源码摘录 20：G17、开漏电气属性和异步边界（constraints/board.xdc:6–34）

```

6: # J7 pin 1 / DEBUG_1. The externally-powered DS18B20 module provides its own
7: # 10 kohm pull-up to J7 pin 20 (+3.3 V), so no internal pull-up is enabled.
8: set_property PACKAGE_PIN G17 [get_ports ds18b20_dq_io]
9: set_property IOSTANDARD LVCMOS33 [get_ports ds18b20_dq_io]
10: set_property DRIVE 4 [get_ports ds18b20_dq_io]
11: set_property SLEW SLOW [get_ports ds18b20_dq_io]
12:
13: set_property PACKAGE_PIN AD12 [get_ports { i_sys_clk_p }]
14: set_property IOSTANDARD DIFF_HSTL_II_18 [get_ports { i_sys_clk_p }]
15: set_property PACKAGE_PIN AD11 [get_ports { i_sys_clk_n }]
16: set_property IOSTANDARD DIFF_HSTL_II_18 [get_ports { i_sys_clk_n }]
17:
18: # UART RX is asynchronous to clk_soc. Cut only the external-port path into
19: # the ASYNC_REG synchronizer; the synchronizer's internal stage-to-stage path
20: # remains timed. UART TX and the debug displays have no synchronous external
21: # sampling clock on the contest board, so their output paths are explicitly
22: # declared asynchronous board interfaces.
23: set_false_path -from [get_ports i_uart_rx] \
24:   -to [get_pins {u_soc/u_peripherals/u_uart/rx_sync_q_reg[0]/D}]
25: set_false_path -to [get_ports o_uart_tx]
26: set_false_path -to [get_ports {virtual_led[*]}]
27: set_false_path -to [get_ports {virtual_seg[*]}]
28: # The 1-Wire slave response is asynchronous at the pad. Only the path into
29: # the first ASYNC_REG stage is cut; the synchronizer and all 50 MHz island
30: # logic remain timed. DQ output-enable timing is protocol based, not source
31: # synchronous to an external clock.
32: set_false_path -from [get_ports ds18b20_dq_io] \
33:   -to [get_pins {u_temperature_island/u_sensor/dq_sync_q_reg[0]/D}]
34: set_false_path -to [get_ports ds18b20_dq_io]

```

8.5 1-Wire 状态机与微秒时序

阶段	本工程时序	实现动作
上电等待	10 ms	等待电源和传感器稳定
空闲检查	连续高 8 μ s	释放 DQ 后确认没有短路或接错脚
Reset	拉低 500 μ s	超过数据手册 480 μ s 下限
Presence	释放后 70 μ s 采样；总恢复 500 μ s	低电平表示设备存在
写 1 / 写 0	70 μ s slot; 低 6 / 60 μ s	LSB first 发送命令
读 bit	低 3 μ s; 第 15 μ s 采样; 70 μ s slot	同步后的 DQ 写入当前 bit
温度转换	释放 DQ 800 ms	外部供电模式，不使用强上拉
故障重试	100 ms	总线低、无 Presence、CRC 错或全零后重试

源码摘录 21：写时隙与读时隙的开漏控制（rtl/periph/ds18b20_master.sv:96–107）

```

96: always_comb begin
97:   dq_drive_low_o = 1'b0;
98:   unique case (state_q)
99:     ST_RESET_LOW: dq_drive_low_o = 1'b1;
100:    ST_WRITE_SLOT: begin
101:      // 70 us slot: write-1 is low for 6 us, write-0 for 60 us.
102:      dq_drive_low_o = tx_bit ? (us_count_q < 6) : (us_count_q < 60);
103:    end
104:    ST_READ_SLOT: begin
105:      // Initiate a read slot for 3 us, then release for the slave response.
106:      dq_drive_low_o = (us_count_q < 3);
107:    end

```

源码摘录 22：Reset、Presence 采样和命令起点（rtl/periph/ds18b20_master.sv:186–217）

```

186: ST_RESET_LOW: begin
187:   // A 500 us reset comfortably exceeds the 480 us minimum.
188:   if (us_count_q == 499) begin
189:     us_count_q <= 32'b0;
190:     state_q <= ST_RESET_RELEASE;
191:     presence_seen_q <= 1'b0;
192:   end else begin
193:     us_count_q <= us_count_q + 1'b1;
194:   end
195: end
196:
197: ST_RESET_RELEASE: begin
198:   // Sample 70 us after release, in the specified presence window.
199:   if (us_count_q == 69) begin
200:     presence_seen_q <= !dq_sync_q[1];
201:   end
202:
203:   // Leave a full 500 us recovery interval before the first slot.
204:   if (us_count_q == 499) begin
205:     us_count_q <= 32'b0;
206:     if (presence_seen_q) begin
207:       sensor_present_o <= 1'b1;
208:       tx_byte_q <= 8'hcc; // Skip ROM: exactly one attached sensor.
209:       bit_index_q <= 3'b0;
210:       command_step_q <= reset_for_read_q ? CMD_READ_SKIP
211:                                           : CMD_CONVERT_SKIP;
212:       state_q <= ST_WRITE_SLOT;
213:     end else begin
214:       sensor_present_o <= 1'b0;
215:       fault_o <= 1'b1;
216:       fault_code_o <= FAULT_NO_PRESENCE;
217:       state_q <= ST_RETRY_WAIT;

```

一次正常循环依次发送 Reset → Skip ROM(0xCC) → Convert T(0x44)，等待 800 ms 后再次发送 Reset → Skip ROM(0xCC) → Read Scratchpad(0xBE)，再读 9 字节。该板只接一个传感器，因此使用 Skip ROM；从上电等待结束后的稳态更新间隔约 0.82 s。

8.6 Scratchpad 校验与温度发布

状态机把前 8 个 scratchpad 字节逐字节送入 Dallas CRC8，反馈多项式以反射形式 0x8C 实现；第 9 字节必须等于累计 CRC。完整 9 字节全零即使 CRC 数值也为零，仍被单独拒绝，避免 DQ 卡低被误显示成 0.00°C。只有 CRC 正确且数据非全零时，scratchpad[1:0] 才作为有符号 1/16°C 原子发布，同时 sample_valid 脉冲一个 50 MHz 周期。

源码摘录 23：9 字节读取、CRC 与原子发布 (rtl/periph/ds18b20_master.sv:268–314)

```

268:         ST_READ_SLOT: begin
269:             // Sample 15 us from the start of the slot, before the 1-Wire limit.
270:             if (us_count_q == 14) begin
271:                 rx_byte_q[bit_index_q] <= dq_sync_q[1];
272:             end
273:
274:             if (us_count_q == 69) begin
275:                 us_count_q <= 32'b0;
276:                 if (bit_index_q == 3'd7) begin
277:                     bit_index_q <= 3'b0;
278:                     if (byte_index_q < 4'd8) begin
279:                         scratchpad_q[byte_index_q] <= rx_byte_q;
280:                         crc_q <= crc8_byte(crc_q, rx_byte_q);
281:                         byte_index_q <= byte_index_q + 1'b1;
282:                         rx_byte_q <= 8'b0;
283:                     end else begin
284:                         state_q <= ST_VALIDATE;
285:                     end
286:                 end else begin
287:                     bit_index_q <= bit_index_q + 1'b1;
288:                 end
289:             end else begin
290:                 us_count_q <= us_count_q + 1'b1;
291:             end
292:         end
293:
294:         ST_VALIDATE: begin
295:             // The ninth byte is the transmitted Dallas CRC of bytes 0..7.
296:             // A nine-byte all-zero frame also has a zero CRC, but is not a
297:             // legal powered DS18B20 scratchpad. Reject it explicitly so an
298:             // electrical-low bus can never be published as 0.00 C.
299:             if ((crc_q == rx_byte_q) && !scratchpad_all_zero) begin
300:                 temperature_raw_o <= $signed({scratchpad_q[1], scratchpad_q[0]});
301:                 sample_valid_o <= 1'b1;
302:                 sensor_present_o <= 1'b1;
303:                 fault_o <= 1'b0;
304:                 fault_code_o <= FAULT_NONE;
305:                 us_count_q <= 32'b0;
306:                 reset_for_read_q <= 1'b0;
307:                 presence_seen_q <= 1'b0;
308:                 state_q <= ST_IDLE_CHECK;
309:             end else begin
310:                 fault_o <= 1'b1;
311:                 fault_code_o <= scratchpad_all_zero ? FAULT_ALL_ZERO : FAULT_CRC;
312:                 us_count_q <= 32'b0;
313:                 state_q <= ST_RETRY_WAIT;
314:             end

```

故障码	触发条件	SEG 显示
1	FPGA 释放输出后 DQ 仍低	dLo
2	Reset 后没有 Presence	noPr
3	第 9 字节与 Dallas CRC8 不符	CrC
4	完整 scratchpad 为全零	0Err

8.7 温度换算与 8 位 SEG 扫描

源码摘录 24：1/16°C 到 0.01°C 的无 DSP 四舍五入 (rtl/periph/temperature_seg_display.sv:68–76)

```

68:     always_comb begin
69:         magnitude_comb = temperature_raw_i[15]
70:             ? (~temperature_raw_i + 1'b1) : temperature_raw_i;
71:         // magnitude * 25 + 2, expressed structurally to keep the display island
72:         // out of the CPU's DSP pool. The final >>2 performs rounded /4.
73:         centi_scaled_comb = ({4'b0, magnitude_comb} << 4)
74:             + ({4'b0, magnitude_comb} << 3)
75:             + {4'b0, magnitude_comb} + 2;
76:     end

```

显示端对有符号原始值取绝对值，以移位加法计算 $\text{magnitude} \times 25 + 2$ ，再右移 2 位得到四舍五入的百分之一摄氏度，不占用 CPU 的 DSP。14 次 double-dabble 把二进制百分之一度转换为 BCD；27.6875°C 得到 27.69，32.375°C 得到 32.38。负温度显示“-5.25”或“-10.1”，100°C 以上显示“125.0”。

板上 8 位数码管由四个双位组组成，XDC 信号组顺序是 LED1、LED2、LED4、LED3，实测物理左到右顺序是 LED3→LED4→LED2→LED1。显示内容固定居中到物理第 2–5 位，其他位保持全灭，不再复制成“3832.3832”。扫描相位每 1 ms 翻转，完整两相刷新率 500 Hz；位选切换时插入 1 个 50 MHz 全灭周期，减少前一位段码残留造成的重影。

源码摘录 25：SEG 物理位序与切换空白 (rtl/periph/temperature_seg_display.sv:267–299)

```

267: // virtual_seg group layout from the board XDC:
268: // [9:0]=LED1, [19:10]=LED2, [29:20]=LED4, [39:30]=LED3.
269: // In each group [7:0]={D,P,G,F,E,D,C,B,A}, [8]=CS1, [9]=CS2.
270: // Physical left-to-right order measured on the board is:
271: // LED3 (digits 0/1), LED4 (2/3), LED2 (4/5), LED1 (6/7).
272: seg_o = 40'b0;
273: seg_o[37:30] = scan_phase_q ? display_digit1_segments
274:                  : display_digit0_segments;
275: seg_o[27:20] = scan_phase_q ? display_digit3_segments
276:                  : display_digit2_segments;
277: seg_o[17:10] = scan_phase_q ? display_digit5_segments
278:                  : display_digit4_segments;
279: seg_o[7:0] = scan_phase_q ? display_digit7_segments
280:                  : display_digit6_segments;
281:
282: // The board's common-cathode digit selects are active low. Insert a
283: // one-clock all-off interval whenever the selected digit changes.
284: seg_o[9:8] = 2'b11;
285: seg_o[19:18] = 2'b11;
286: seg_o[29:28] = 2'b11;
287: seg_o[39:38] = 2'b11;
288: if (!scan_blank_q) begin
289:   if (!scan_phase_q) begin
290:     seg_o[8] = 1'b0;
291:     seg_o[18] = 1'b0;
292:     seg_o[28] = 1'b0;
293:     seg_o[38] = 1'b0;
294:   end else begin
295:     seg_o[9] = 1'b0;
296:     seg_o[19] = 1'b0;
297:     seg_o[29] = 1'b0;
298:     seg_o[39] = 1'b0;
299:   end

```

8.8 同一 bit 文件内的功能整合

jyd_soc_top 在同一个顶层和同一个 bit 文件中实例化两条电路：clk_out2 驱动 150 MHz CPU、Cache、BRAM、UART、Timer、IRQ 和 RT-Thread；clk_out1 固定驱动 50 MHz DS18B20 与 SEG。两个时钟域各有独立的复位同步释放链。测温链没有 CPU bus、IRQ、软件变量或 clk_soc 输入，因此系统运行 CoreMark、MSH 输出或 Cache miss 时，1-Wire 微秒计数与 SEG 扫描仍按 50 MHz 连续运行。

源码摘录 26：同一顶层中的双时钟功能连接 (rtl/soc/jyd_soc_top.sv:36–52, 84–95)

```

36: // The only reused IP block is the official active Clocking Wizard XCI.
37: pll u_pll (
38:   .clk_in1_p(i_sys_clk_p),
39:   .clk_in1_n(i_sys_clk_n),
40:   .clk_out1(clk_board_50m),
41:   .clk_out2(clk_soc),
42:   .locked(pll_locked)
43: );
44:
45: // PLL lock loss asserts reset asynchronously. Each used clock domain owns
46: // its own two-flop synchronous deassertion chain.
47: reset_sync u_soc_reset (
48:   .clk_i(clk_soc), .arst_ni(pll_locked), .rst_no(rst_n)
49: );
50:
51: reset_sync u_board_reset (
52:   .clk_i(clk_board_50m), .arst_ni(pll_locked), .rst_no(board_rst_n)
53: );
54: // synchronizer, the sensor pin, and all SEG pins. No CPU bus, software state,
55: // or clk_soc signal enters this island, so system load cannot stall sampling
56: // or display refresh and there is no inter-domain data CDC to constrain.
57: temperature_display_island #(
58:   .CLK_HZ(50_000_000)
59: ) u_temperature_island (
60:   .clk_i(clk_board_50m),
61:   .rst_ni(board_run_q),
62:   .dq_i(ds18b20_dq_in),

```

```

93:     .dq_drive_low_o(ds18b20_drive_low),
94:     .seg_o(virtual_seg)
95: );

```

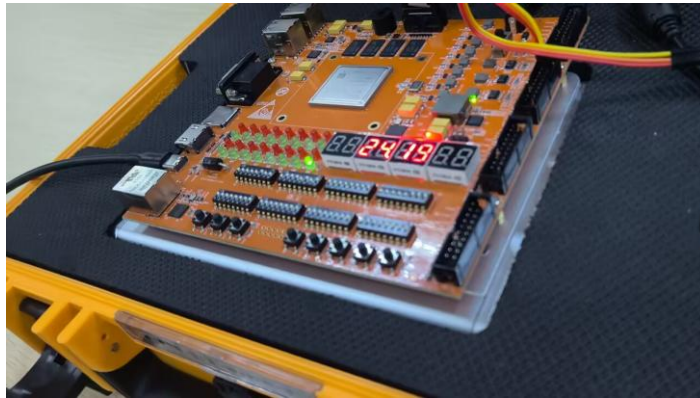


图 9 同一 bit 文件运行 RT-Thread SoC 与 50 MHz 温度显示 (约 24.19°C)

9 从小模块到实板的验证

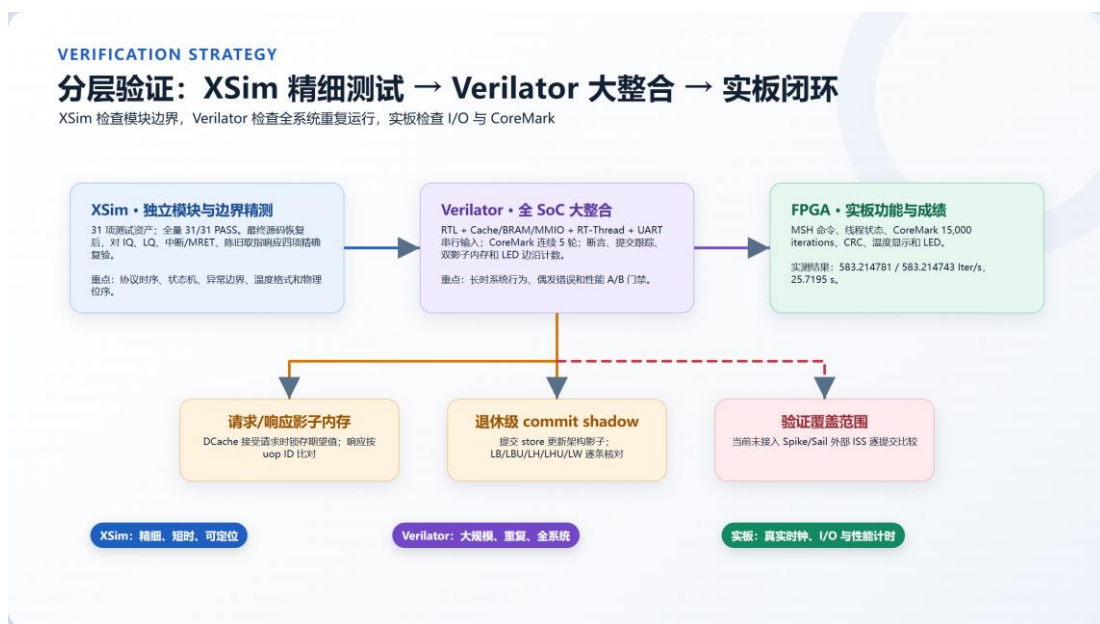


图 10 XSim、Verilator 与实板的分层验证体系

9.1 XSim 独立模块与边界精测

测试脚本为每个 top 创建独立目录, 依次运行 xvlog → xelab → xsim, 同时检查工具退出码、Fatal/Error 和 PASS <top> 签名 [7]。工程共有 31 项模块测试资产, 完整 31/31 运行通过; 恢复产生 WNS +0.001 ns bit 的源码后, 又对直接受影响的 issue queue、load queue、interrupt/MRET 边界和陈旧取指响应四项进行了精确复验。

```

XSim 2025.2 • 31 项模块测试

PS> .\sim\run_xsim_unit_tests.ps1

PASS tb_branch_checkpoints      PASS tb_branch_predictor
PASS tb_crc16_accelerator       PASS tb_crc32_word_accelerator
PASS tb_csr_execute            PASS tb_csr_file
PASS tb_data_cache             PASS tb_dmem_request_fifo
PASS tb_dmem_response_collision PASS tb_ds18b20_master
PASS tb_fetch_bundle_queue     PASS tb_fetch_redirect_stale_response
PASS tb_free_bitmap            PASS tb_instruction_cache
PASS tb_issue_queue            PASS tb_load_queue
PASS tb_mmio_decode            PASS tb_mret_interrupt_boundary
PASS tb_muldiv_unit            PASS tb_physical_regfile
PASS tb_rename_map             PASS tb_rob
PASS tb_rv32_agu_add_imm       PASS tb_rv32_alu
PASS tb_rv32_decode            PASS tb_rv32_mul_partial_products
PASS tb_state_transition_accelerator
PASS tb_store_queue            PASS tb_temperature_seg_display
PASS tb_tick_timer             PASS tb_uart

31 / 31 module tests passed

Exact +0.001 ns focused rerun:
PASS tb_issue_queue
PASS tb_load_queue
PASS tb_mret_interrupt_boundary cycles=70 mepc=00000040 mcause=80000007
PASS tb_fetch_redirect_stale_response

```

图 11 XSim 31 项模块回归与四项重点复验终端摘录

9.2 Verilator 全 SoC 大整合

大整合回归将完整 CPU、I/D Cache、64 KiB BRAM、MMIO、UART、Timer 和 RT-Thread 固件编译到 Verilator C++ 模型，并启用 --assert [12]。测试平台不直接写 UART MMIO，而是通过真实 RX 串行波形输入 help、ps、list_sem、simple_add、status、coremark 和 Team ID。它同时统计 fetch/dispatch/issue/retire、分支、Cache 和阻塞计数器，验证 CRLF、中断快照、每次 CoreMark LED 一次上升/一次下降，并连续运行 CoreMark 5 次以防偶发性错误。

```

Verilator • 全 SoC + RT-Thread + CoreMark 五轮回归

[PERF] active_cycles=2467209 retired=2652241 native_ipc=1.074996 effective_ipc=1.206317
[PERF] issue0=1457086 issue1=1491784 issue_both=933590
[PERF] dcache_load=564805 dcache_hit=564800 dcache_miss=3
[PERF] branch_resolve=635418 branch_mispredict=68338

CoreMark Size      : 666
Total ticks        : 9637
Total time (secs): 0.016447
Iterations/Sec     : 608.008198
Iterations         : 10
[0]crclist         : 0xe714
[0]crcmatrix       : 0x1fd7
[0]crcstate        : 0x8e3a
[0]crcfinal        : 0xfcaf
COREMARK DONE rc=0 runs=5

PASS tb_soc_firmware cycles=254137833 led_rises=5 led_falls=5 traps=11 mret=24

Five-run range:
  Iterations/Sec  607.945113 - 608.008198
  effective IPC   1.206317 - 1.206352
  fatal trap      none
  CRC mismatch    none

```

图 12 Verilator 第五次 CoreMark 与全系统 PASS 摘录

9.3 访存和退休结果逐项核对

第一份结果副本在 DCache 接受写请求时更新字节，在 load 发出时保存正确值，收到响应后按 uop ID 比较。第二份结果副本只记录已经退休的 store，再按地址、数据大小和符号扩

展算出每条退休 load 应得到的值，并与 commit_trace.rd_data 比较。两份副本都由同一份固件 BRAM 初始化，分别检查访存请求/响应和最终退休结果。

源码摘录 27：退休 load 与结果副本断言（sim/verilator/tb_soc_firmware.sv:609–621）

```
609:         if ({load_addr < RAM_BYTES} &&
610:             ((compare_mask & {
611:                 commit_shadow_valid[{load_addr[RAM_ADDR_BITS-1:2], 2'b00} + 3],
612:                 commit_shadow_valid[{load_addr[RAM_ADDR_BITS-1:2], 2'b00} + 2],
613:                 commit_shadow_valid[{load_addr[RAM_ADDR_BITS-1:2], 2'b00} + 1],
614:                 commit_shadow_valid[{load_addr[RAM_ADDR_BITS-1:2], 2'b00} + 0}]) ==
615:                 compare_mask)) begin
616:             assert (dut.commit_trace[lane].rd_data == expected_value)
617:         else $fatal(1,
618:             "committed load mismatch pc=%08x addr=%08x size=%0d expected=%08x
actual=%08x sp=%08x",
619:             dut.commit_trace[lane].pc, load_addr, load_size,
620:             expected_value, dut.commit_trace[lane].rd_data,
621:             committed_register_value(2));
```

差分检查范围

commit trace 和两份影子内存分别检查请求/响应级与退休级存储一致性。当前验证未接入 Spike、Sail、QEMU 或随机指令生成器，外部 ISS 逐提交比较不在本轮验证范围内。

10 CoreMark 性能与 IPC



图 13 IPC、加速器与频率的联合优化方法

10.1 性能提升策略

方向	采取措施	性能/时序权衡
前端	8 请求在途、8 项 fetch bundle queue、预测元数据跟随请求	降低 IS 延迟暴露；用 epoch 阻止陈旧响应污染
窗口	双 8 项 IQ、oldest-ready、5 个持久 wakeup + 3 个 select-wakeup	曾试 16 项/队列，但 age matrix 和广播扩大的物理代价超过 CoreMark 收益，因此回退

方向	采取措施	性能/时序权衡
PRF/WB	LUTRAM 按 lane/WB 来源复制，EX/WB 局部边界完成旁路合并	降低全局扇出，但不盲目新增架构流水级
LSU	窄 payload、bit16 分段 AGU、dmem FIFO、LQ 依赖 bitmap、RAM fast lane	切断 ready 反馈与宽 selector→DS 链，同时保留常规 load 吞吐
加速器	CRC16/CRC32-word/state transition Custom-0	有效工作量提升计入 effective IPC；必须保持 CRC 和透明报告编译标志

后续可考虑指令融合，在不改变 RISC-V 架构 ISA 的前提下减少内部操作数，但任何融合都必须保留异常 PC、提交语义和逐指令可观测性 [18]。协处理器/大小核设计继续暂缓，直到参赛负载稳定，避免为不确定负载扩张状态和局部互连。

10.2 Verilator 五次重复性

轮次	active cycles	native IPC	effective IPC	Iter/s (回归)
1	2,467,138	1.075027	1.206352	608.008198
2	2,467,209	1.074996	1.206317	608.008198
3	2,467,209	1.074996	1.206317	607.945113
4	2,467,209	1.074996	1.206317	608.008198
5	2,467,209	1.074996	1.206317	608.008198

native IPC 是该轮实际退休指令数除以 active cycles。effective IPC = $2,976,237 \div \text{accelerated active_cycles}$ ，其中 2,976,237 是未加速 10-iteration 参考测试的退休工作量；它用于比较“完成同等软件工作量”的效率，不等同于架构实际退休 IPC。五次均通过 e714/1fd7/8e3a/fcaf，无 fatal trap，LED rises/falls 均为 5。

10.3 实板加速 CoreMark

```
add 1 to 999, sum=499500
msh >ps
thread  pri  status  sp  stack size max used left tick  error
-----
coremark  5  suspend 0x000000f0 0x00002000 02% 0x00000009 000
tshello  20  ready  0x00000000 0x00000000 16% 0x00000009 000
tidle    31  ready  0x00000000 0x00000200 25% 0x00000020 000
msh >coremark
=====
Please enter your Team ID and press Enter:
team id: CICC1004534
Team ID locked: CICC1004534
Starting CoreMark, please wait...
=====
CoreMark Team ID: CICC1004534
COREMARK BEGIN (RT-Thread worker, stack=8192)
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 15070027
Total time (secs): 25.719513
Iterations/Sec : 583.214781
Iterations : 15000
Compiler version : Clang 22.1.7 (https://github.com/msys2/MINGW-packages b8f306ad57e0c8f3f15aed0dc809d7e3e8e2eee9)
Compiler flags : -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1
Memory location : STATIC
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crctrinal : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 583.214781 / Clang 22.1.7 (https://github.com/msys2/MINGW-packages b8f306ad57e0c8f3f15aed0dc809d7e3e8e2eee9) -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1 / STATIC
CoreMark Team ID: CICC1004534
COREMARK DONE rc=0 runs=1
msh >coremark
=====
Please enter your Team ID and press Enter:
team id: CICC1004534
Team ID locked: CICC1004534
Starting CoreMark, please wait...
=====
CoreMark Team ID: CICC1004534
COREMARK BEGIN (RT-Thread worker, stack=8192)
```

图 14 实板第一轮 15,000 iterations CoreMark 与第二轮启动

```
Compiler flags : -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1
Memory location : STATIC
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crctrinal : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 583.214781 / Clang 22.1.7 (https://github.com/msys2/MINGW-packages b8f306ad57e0c8f3f15aed0dc809d7e3e8e2eee9) -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1 / STATIC
CoreMark Team ID: CICC1004534
COREMARK DONE rc=0 runs=1
msh >coremark
=====
Please enter your Team ID and press Enter:
team id: CICC1004534
Team ID locked: CICC1004534
Starting CoreMark, please wait...
=====
CoreMark Team ID: CICC1004534
COREMARK BEGIN (RT-Thread worker, stack=8192)
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 15070028
Total time (secs): 25.719514
Iterations/Sec : 583.214743
Iterations : 15000
Compiler version : Clang 22.1.7 (https://github.com/msys2/MINGW-packages b8f306ad57e0c8f3f15aed0dc809d7e3e8e2eee9)
Compiler flags : -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1
Memory location : STATIC
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crctrinal : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 583.214743 / Clang 22.1.7 (https://github.com/msys2/MINGW-packages b8f306ad57e0c8f3f15aed0dc809d7e3e8e2eee9) -O3 -march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1 / STATIC
CoreMark Team ID: CICC1004534
COREMARK DONE rc=0 runs=2
msh >ticks
ticks command not found.
msh >ticks
TICK 6
msh >
```

图 15 实板第二轮 CoreMark 与 ticks 命令

轮次	Iterations	时间	Iterations/Sec	CRC
1	15,000	25.719513 s	583.214781	e714 / 1fd7 / 8e3a / 65c5
2	15,000	25.719514 s	583.214743	e714 / 1fd7 / 8e3a / 65c5

构建参数 (software/build_firmware.ps1:35-46, 126)

```
--target=riscv32-unknown-elf -O3 -ffreestanding -fno-builtin -fno-common
-fno-stack-protector -msmall-data-limit=0 -ffunction-sections -fdata-sections
-march=rv32im_zba_zicsr_zifencei -mabi=ilp32 -DJYD_COREMARK_HW_ACCEL=1
```

内存位置 STATIC，Team ID=CICC1004534。官方 CoreMark 核心算法文件文本与发布包一致；两轮均显示 Correct operation validated，固定 15,000 iterations、CRC 一致，LED 覆盖性能计时窗口 [23]。

11 150 MHz 时序优化与最终布线



图 16 从综合到最终 bit DCP 的 WNS 变化

11.1 每个阶段的建立与保持时间

阶段	WNS	TNS	setup 负端点	WHS	THS	hold 负端点
synth	-1.149	-136.036	283	-0.046	-198.186	11,497
opt	-1.378	-156.261	478	-0.046	-198.186	11,497
place	-0.513	-72.118	505	-0.116	-9.898	348
pre-route physopt	0.000	0.000	0	-0.052	-5.557	280
route	-0.224	-7.729	100	+0.045	0.000	0
post-route physopt	-0.099	-1.766	51	+0.045	0.000	0
reroute	-0.097	-1.292	40	+0.045	0.000	0
本轮 post-route physopt	-0.001	-0.001	1	+0.045	0.000	0
final bit DCP	+0.001	0.000	0	+0.045	0.000	0

表中数据来自 metadata/STAGE_TIMING_SUMMARY_20260815.md 和 timing/stages/all_stage_raw_setup_hold_reports.zip。

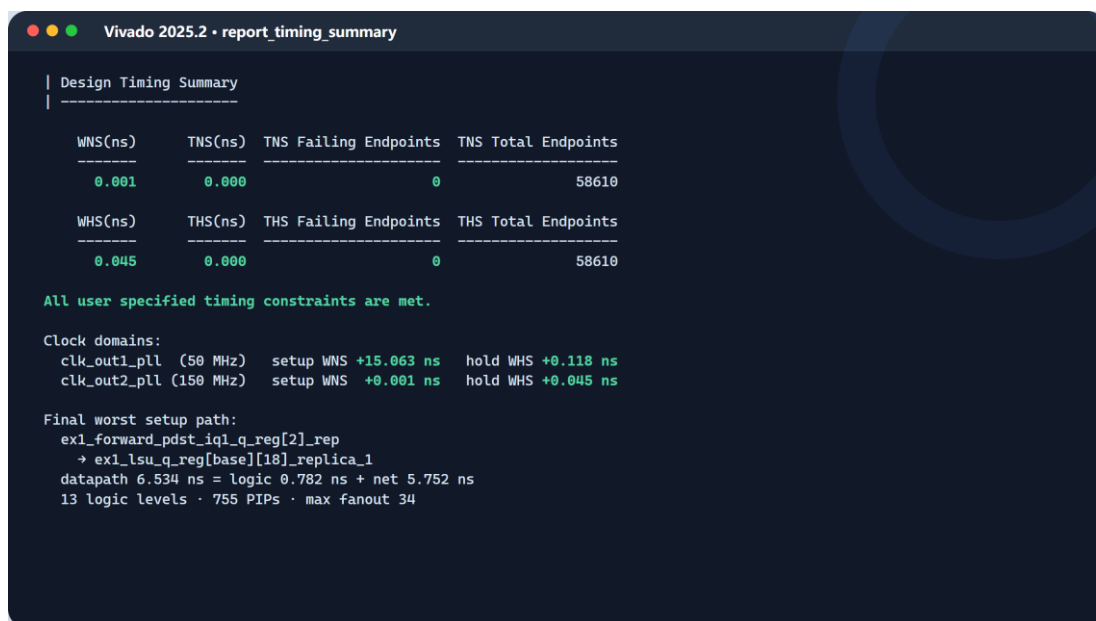


图 17 生成该 bit 文件的最终 DCP 时序结果

11.2 时序收敛思路

- 先检查串行优先级逻辑、宽扇出、组合反馈和重复计算，再决定是否加入寄存边界。流水级只放在具有明确握手、停顿和恢复语义的位置。
- 每一阶段同时拉取 setup 和 hold。路由器修 hold 可能增加 setup 数据路径延迟，不能只看 WNS 或只看 setup 条数。
- 对每条紧路径记录 start/end cell、SLICE 坐标、logic/net delay 比例、PIP/node 数和 fanout，区分逻辑深度问题与局部绕路问题 [10][11]。
- 用相同网表、约束和 checkpoint 对 HigherDelayCost、AggressiveExplore 和 post-route phys_opt 进行 A/B，不保留使 WNS/TNS/负端点整体恶化的版本 [9]。
- 资源总占用率不能代替局部物理判断。IQ/PRF/ROB 的 SLICEM/LUTRAM、BRAM 固定列、高连接度 EX/LSU 会在局部资源尚未“全满”时提前造成通道紧张。

11.3 最终最慢路径

属性	结果
起点 → 终点	ex1_forward_pdst_iq1_q_reg[2]_rep → ex1_lsu_q_reg[base][18]_replica_1
slack / datapath	+0.001 ns / 6.534 ns
logic / net	0.782 ns / 5.752 ns (约 88%)
logic levels	13
坐标	SLICE_X17Y225 → SLICE_X36Y207, span 19×18
布线	16 条数据网、755 PIP, 单网最大 287 PIP, 最大 fanout 34

这条路径的逻辑延迟约占 12%，其余延迟主要来自 IQ、PRF、EX 和 LSU 之间的局部物理互连。复制操作数状态会增加局部连接数量，未形成握手边界时不能缩短原组合路径。最终网表保留局部物理优化，未保留大范围操作数复制。

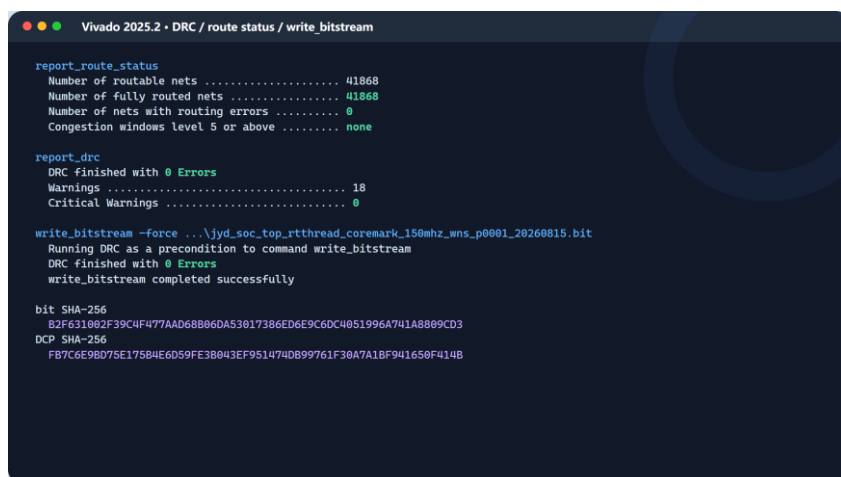


图 18 route status、DRC 与 write_bitstream 成功摘录

11.4 资源使用

层级	LUT	Logic LUT	LUTRAM	FF	RAMB36	DSP
全芯片	33,372	29,904	3,468	11,387	24	4
u_soc	33,028	29,560	3,468	11,144	24	4
u_core	26,086	22,954	3,132	8,053	0	4
独立测温显示电路 (u_temperature_island)	343	343	0	237	0	0

12 最终结果、复现步骤与已知问题

12.1 最终实现结果

- report_timing_summary: 所有用户时序约束满足, setup/hold 全量负路径均为 No timing paths found。
- check_timing : no_clock=0 、 unconstrained_internal_endpoints=0 、 loops=0 、 partial_input/output_delay=0、latch_loops=0。
- 板级 I/O 报告包含 no_input_delay=2、no_output_delay=27。内部寄存器路径均已受约束; 外部器件接口需要在整机时序检查中补充板外延迟预算。
- DRC 0 Error; 18 项 Warning 为 DSP pipeline、BUFG 非最优连接和无 routable load。Methodology 88 项 Warning 为 RAM 推断/映射提示, 无 Error/Critical Warning。
- 41,868/41,868 可布线网络 fully routed, routing error=0, 无 level 5 以上拥塞窗口。

12.2 复现步骤

复现时先解压源码归档 source/jyd2025_m0097_source_snapshot_20260815_1217.zip, 再用 Vivado 2025.2 执行 flow/create_project.tcl 生成项目副本。目标器件为 xc7k325tffg900-2,

PLL clk_out2=150 MHz、clk_out1=50 MHz。Clang/RISC-V 工具链生成固件并写入 BRAM init。RTL 或固件修改后，需要重新运行 XSim 重点测试、Verilator 全 SoC 五轮回归、综合至最终 route 的全阶段 setup/hold、DRC 和 bitgen。最终 DCP 必须与对应源码快照配套使用。

12.3 已知问题与后续工作

限制	影响	建议
外部 ISS difftest 未建立	当前存储差分与断言很强，但不覆盖所有指令/特权角落	引入 Sail/Spike 逐提交参考、随机指令和异常停止信息 [19][22]
predictor checkpoint row 别名风险	少数 bank 偏斜模式可能恢复错的 GHR/RAS 快照	增加 row 4-6 定向测试并统一 snapshot 索引容量
板级 I/O delay 未完整	不影响内部 STA，但外部器件系统级时序未完整建模	根据 UART、SEG、DS18B20 和板级负载补充预算
支持 ISA 有限	不支持 RVC/A/F/D/S/U	只在赛题负载明确需要时扩展，避免无效面积和布线成本
正时序裕量很小	+0.001 ns 会对布局种子、工具版本、温度、电压和源码改动敏感	保存产生该 bit 的最终 DCP；新改动须重跑全流程与实板重复

附录 A 源码文件说明

Top rtl/soc/jyd_soc_top.sv: PLL 双输出、双复位链、SoC、独立测温显示电路和 CoreMark LED

Frontend

rtl/core/frontend/{branch_predictor,fetch_bundle_queue}.sv: BTB/PHT/RAS、GHR、预测更新、64-bit 包队列与半包前移

Rename/PRF

rtl/core/rename/{rename_map,free_bitmap,branch_checkpoints,physical_regfile*.sv: RAT/RRAT、物理寄存器分配、分支快照、4 读 2 写 PRF

Backend rtl/core/backend/{issue_queue,rob}.sv: oldest-ready、wakeup/select-wakeup、乱序完成与按序提交

Execute

rtl/core/execute/{rv32_alu,rv32_agu_add_imm,muldiv_unit,*accelerator}.sv: ALU、分段 AGU、乘除法与 CoreMark 算法等加速

LSU/Cache rtl/core/lsu/{load_queue,store_queue}.sv: rtl/cache/{instruction_cache,data_cache}.sv: 依赖与 forwarding、store drain、I\$ refill、D\$ 2 MSHR 和回写

SoC

rtl/soc/{ooo_soc_system,dmem_request_fifo,unified_bram}.sv: 总线路由、请求边界、双口主存

MMIO rtl/{periph/uart,periph/tick_timer,soc/irq_debug}.sv: 串口、节拍、IRQ/BUILD_ID/LED

Temperature

rtl/periph/{ds18b20_master,temperature_seg_display,temperature_display_island}.sv: 1-Wire、CRC8、格式化和物理 SEG 扫描

RTOS software/rththread/{start,board,app,msh_demo}.*: 启动、trap/BSP、MSH 命令和 CoreMark worker

Verify sim/run_xsim_unit_tests.psl: sim/run_verilator_firmware_tests.psl: sim/verilator/tb_soc_firmware.sv: 31 项 XSim 模块测试、全 SoC 固件回归、影子存储与 LED 边沿

附录 B 工具和版本

工具/平台	版本/配置	用途
AMD Vivado / XSim	2025.2 Build 6299465	综合、实现、STA、bitgen 与精细模块测试
Verilator	5.048, --assert	全 SoC 大整合、重复回归与计数器
Clang	22.1.7, -O3, rv32im zba zicsr zifencei/ilp32	RT-Thread 与 CoreMark 固件
RT-Thread	Nano 3.1.5 + FinSH/MSH	系统调度、命令行和 CoreMark worker
FPGA	XC7K325T-2 FFG900	150 MHz SoC + 50 MHz 温度显示

参考文献

- [1] RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture. Version 20260120, 2026. [资料链接](#)
- [2] RISC-V International, The RISC-V Instruction Set Manual, Volume II: Privileged Architecture. Version 20260120, Machine ISA 1.13, 2026. [资料链接](#)
- [13] R. M. Tomasulo, An Efficient Algorithm for Exploiting Multiple Arithmetic Units. IBM Journal of Research and Development, 11(1):25–33, 1967. DOI 10.1147/rd.111.0025. [资料链接](#)
- [14] J. E. Smith and A. R. Pleszkun, Implementing Precise Interrupts in Pipelined Processors. IEEE Transactions on

- [3] RT-Thread Development Team, RT-Thread v3.1.5 Release and Source Tree. Official release, 2021. [资料链接](#)
- [4] RT-Thread Development Team, FinSH Console. RT-Thread Programming Manual and v3.1.5 source. [资料链接](#)
- [5] EEMBC, CoreMark 1.0 Source, Run Rules and Reporting Rules. Official benchmark repository. [资料链接](#)
- [6] Analog Devices (Maxim Integrated), DS18B20 Programmable Resolution 1-Wire Digital Thermometer. Data Sheet, Rev. 6, 2019. [资料链接](#)
- [7] AMD, Vivado Design Suite User Guide: Logic Simulation (UG900). Version 2025.2, 2025. [资料链接](#)
- [8] AMD, Vivado Design Suite User Guide: Using Constraints (UG903). Version 2025.2, 2025. [资料链接](#)
- [9] AMD, Vivado Design Suite User Guide: Implementation (UG904). Version 2025.2, 2025. [资料链接](#)
- [10] AMD, Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906). Version 2025.2, 2025. [资料链接](#)
- [11] AMD, UltraFast Design Methodology Guide for FPGAs and SoCs (UG949). Version 2025.2, 2025. [资料链接](#)
- [12] W. Snyder, Verilator User's Guide. Release 5.050, 2026. [资料链接](#)
- Computers, 37(5):562–573, 1988. DOI 10.1109/12.4607. [资料链接](#)
- [15] J. E. Smith, A Study of Branch Prediction Strategies. ISCA-8, pp.135–148, 1981. DOI 10.1145/800052.801871. [资料链接](#)
- [16] T.-Y. Yeh and Y. N. Patt, Two-Level Adaptive Training Branch Prediction. MICRO-24, pp.51–61, 1991. DOI 10.1145/123465.123475. [资料链接](#)
- [17] S. Palacharla, N. P. Jouppi and J. E. Smith, Complexity-Effective Superscalar Processors. ISCA-24, pp.206–218, 1997. DOI 10.1145/264107.264201. [资料链接](#)
- [18] C. Celio, P. Dabbelt, D. A. Patterson and K. Asanović, The Renewed Case for the Reduced Instruction Set Computer: Avoiding ISA Bloat with Macro-Op Fusion for RISC-V. arXiv:1607.02318, 2016. [资料链接](#)
- [19] RISC-V International/SRI/University of Cambridge, Sail RISC-V Model. Official executable ISA model. [资料链接](#)
- [20] R. Ginosar, Metastability and Synchronizers: A Tutorial. IEEE Design & Test of Computers, 28(5):23–35, 2011. DOI 10.1109/MDT.2011.113. [资料链接](#)
- [21] S. Gal-On and M. Levy, Exploring CoreMark—A Benchmark Maximizing Simplicity and Efficacy. EEMBC white paper, 2009/2012. [资料链接](#)
- [22] RISC-V International, RISC-V Architectural Test Framework. Official architecture test repository. [资料链接](#)
- [23] 竞业达, 2026 集创赛国赛赛题考核点. 比赛方提供文件, 2026; 规定 RT-Thread Nano 3.1.5、CoreMark 15,000 iterations、核心算法文件哈希、CRC、LED 计时及传感器应用要求。